

<!-- Version expliquée, pour débutant, de l'étape 1 du simulateur CMU. -->

Simulateur de parcours assuré CMU

Étape 1/6 — Le schéma PostgreSQL, expliqué pas à pas

Version pédagogique — même contenu technique que `ETAPE_1_GUIDE_COMPLET.md`, mais avec les concepts expliqués au fur et à mesure.

Ce document contient exactement le même code et les mêmes commandes que le guide original. Rien n'a été retiré. Ce qui change : chaque notion technique est expliquée **avant** d'être utilisée, avec des mots simples.

0. Avant de commencer : les concepts qu'il faut connaître

Si tu connais déjà ces notions, passe directement à la section 1. Sinon, lis ceci d'abord — tout le reste du document s'appuie dessus.

C'est quoi une base de données relationnelle ?

Imagine un classeur avec plusieurs tableurs Excel à l'intérieur (des **tables**). Chaque tableur a des colonnes fixes (ex. nom, prénom, date de naissance) et chaque ligne est une **fiche** (un assuré, un centre de santé, une facture...). PostgreSQL est le logiciel qui stocke et gère ce classeur de façon fiable, même avec des milliers de lignes et plusieurs personnes qui lisent/écrivent en même temps.

Table : un tableau avec des colonnes fixes. Ex. `TB_REF_ASSURES` = la table des assurés.

Colonne : une information précise sur chaque ligne (ex. `ASSURE_NOM`).

Ligne (ou "enregistrement") : une fiche complète, ex. un assuré précis.

Clé primaire (`PRIMARY KEY`) : la colonne (ou la combinaison de colonnes) qui identifie une ligne de façon unique — comme un numéro de sécurité sociale. Deux lignes ne peuvent jamais avoir la même clé primaire.

Clé étrangère (`FOREIGN KEY`) : une colonne qui pointe vers la clé primaire d'une autre table, pour créer un lien. Ex. une facture pointe vers l'assuré qui l'a reçue : ça évite de recopier toutes les infos de l'assuré dans chaque facture.

Clé composite : une clé primaire faite de **plusieurs colonnes** à la fois (ex. centre + agent + date de début), utilisée quand aucune colonne seule ne suffit à identifier une ligne de façon unique.

Relation (ou "association") : le lien logique entre deux tables via une clé étrangère. Ex. "un assuré **a plusieurs** factures".

Schéma : la structure complète de la base — la liste des tables, de leurs colonnes, de leurs clés et de leurs relations. **Le schéma ne contient aucune donnée**, seulement la structure vide (comme un tableur Excel dont les colonnes existent mais qui n'a aucune ligne remplie).

C'est quoi un ORM, et pourquoi SQLAlchemy ?

On pourrait écrire directement du SQL (`CREATE TABLE TB_REF_ASSURES (...)`) pour créer les tables. À la place, ce projet utilise un **ORM** (*Object-Relational Mapper*) : une bibliothèque qui permet de décrire les tables comme des classes Python, et qui génère le SQL correspondant toute seule. **SQLAlchemy** est l'ORM utilisé ici. L'avantage : le code Python de l'application (plus tard, l'API) peut manipuler des objets Python normaux (`assure.nom`) plutôt que d'écrire du SQL à la main partout.

C'est quoi une migration, et pourquoi Alembic ?

Une **migration** est un script versionné qui décrit un changement du schéma ("créer telle table", "ajouter telle colonne") — un peu comme un commit Git, mais pour la structure de la base de données. Ça permet de :

- recréer la base de données identique sur un autre ordinateur ;
- suivre l'historique des changements de structure ;
- annuler un changement si besoin (`downgrade`).

Alembic est l'outil qui gère ces migrations pour SQLAlchemy. Ici, la première migration ("révision initiale") va créer les 22 tables d'un coup, à partir des classes Python définies avec SQLAlchemy.

Vocabulaire utilisé plus loin dans ce guide

Terme	Explication simple
<code>async</code> / asynchrone	Le code peut traiter plusieurs opérations "en même temps" sans bloquer, utile quand on attend une réponse de la base de données.
Environnement virtuel (<code>venv</code>)	Un dossier isolé qui contient une version de Python + ses bibliothèques, propre à ce projet, pour ne pas mélanger avec d'autres projets sur ton ordinateur.
<code>UUID</code>	Un identifiant unique très long et quasi impossible à deviner (ex. <code>3fa85f64-5717-...</code>), utilisé ici pour identifier chaque assuré.
Colonnes d'audit	4 colonnes présentes sur <i>chaque</i> table pour savoir qui a créé/modifié une ligne et quand — une bonne pratique de traçabilité.

1. Ce que tu vas obtenir à la fin de cette étape

À la fin, ton projet contiendra :

22 tables PostgreSQL, toutes nommées `TB_XXX` (en majuscules) ;
les **4 colonnes d'audit** sur chaque table (qui a créé/modifié, et quand) ;
les **clés primaires** (simples ou composites) et les **clés étrangères** ;
une **première migration Alembic** qui crée tout ça ;
un script pour créer la base et lancer la migration automatiquement ;
un **diagramme** qui montre visuellement toutes les tables et leurs liens.

Ce que cette étape ne fait PAS (volontairement, ça viendra plus tard) : aucune donnée n'est insérée (étape 2), aucune API, aucune simulation, aucun WebSocket, aucun dashboard.

Une convention de nommage à bien comprendre

Dans PostgreSQL, les noms de tables et de colonnes sont écrits en **MAJUSCULES** : `TB_REF_ASSURES`, `PERSONNE_UUID`, `DATE_CREATION`. C'est le format "physique" imposé par le cahier des charges du projet.

Mais dans le code **Python**, on garde les conventions Python normales : les noms de variables et de classes restent en `snake_case` ou en `CamelCase` (ex. `personne_uuid`, `class InsuredPerson`). SQLAlchemy fait le lien entre les deux automatiquement — c'est pour ça que tu verras souvent, dans le code, une colonne Python en minuscules suivie de son vrai nom PostgreSQL entre guillemets, par exemple :

```
python
personne_uuid: Mapped[UUID] = mapped_column("PERSONNE_UUID", ...)
#          ^ nom Python          ^ nom réel dans PostgreSQL
```

Pourquoi Alembic précisément

Alembic est l'outil de migration "officiel" de SQLAlchemy 2.0 : il connaît directement les classes Python définies plus loin et sait générer/appliquer le SQL correspondant. Il utilisera la même URL de connexion (`asyncpg`) que la future API FastAPI (étape 5) — donc pas de configuration à refaire plus tard.

2. Les choix techniques expliqués

Le cahier des charges du projet donne les noms des tables et des colonnes, mais pas toujours leur type exact (texte ? nombre ? date ?) ni si elles sont obligatoires. Voici les choix faits, et **pourquoi** :

Choix

`UUID` pour `PERSONNE_UUID`

`DATE` pour les dates de validité/soins

`TIMESTAMP WITH TIME ZONE` pour les dates d'audit

`NUMERIC(15,2)` pour les montants en FCFA

`NUMERIC(5,2)` pour les taux

`VARCHAR(3)` pour le code pathologie

`TEXT` pour observations/motifs

`INTEGER` auto-incrémenté pour
`ENTENTE_PREALABLE_ID`

`NOT NULL` sur les clés (primaires et étrangères "métier")

Le reste : nullable

Pourquoi

Identifiant impossible à deviner, adapté à des données sensibles (santé).

On ne garde que le jour, pas l'heure — cohérent avec le métier.

On veut savoir précisément *quand* (heure comprise) une ligne a été créée/modifiée.

Un nombre décimal *exact* (pas d'arrondi flottant hasardeux) — indispensable pour de l'argent.

Idem, mais avec une plage plus petite (un taux ne dépasse pas 100 %).

Le cahier des charges impose 3 caractères exactement.

Texte libre, de longueur non prévisible.

Un simple compteur suffit ici, pas besoin d'UUID.

Une ligne sans identifiant n'a pas de sens.

Sauf mention contraire, on n'invente pas d'obligation que le cahier des charges ne demande pas.

Une règle de prudence importante : le code ne crée **aucune clé étrangère** depuis une colonne "code" seule vers un référentiel dont la clé primaire est composite (code + date). Exemple : `TYPE_FACTURE_CODE` seul ne peut pas pointer proprement vers la clé composite de `TB_TV_TYPES_FACTURES` (qui est "code + date de début"). Créer quand même ce lien produirait une fausse contrainte, techniquement incorrecte. Mieux vaut l'omettre que de mentir sur la structure des données.

3. Le mode opératoire, étape par étape

Cette partie est le cœur du travail pratique : exécute les étapes **dans l'ordre**, et ne passe à la suivante que lorsque le contrôle indiqué réussit. Chaque étape est expliquée avant d'être exécutée.

Étape 1 — Se placer dans le bon dossier

Pourquoi : toutes les commandes suivantes supposent que le terminal est positionné à la racine du projet. Si tu es dans le mauvais dossier, les fichiers seront créés au mauvais endroit (ou les commandes échoueront).

Sous Windows PowerShell :



powershell
Set-Location "C:\Users\charles.nguessan\Documents\simulateur_V3"
Get-Location

Sous Linux ou macOS (adapte le chemin à ton propre dossier) :



bash
cd /chemin/vers/simulateur_V3
pwd

Résultat attendu : le chemin affiché se termine par `simulateur_V3`.

Ne continue pas si le terminal pointe vers ton dossier personnel ou un autre projet.

Étape 2 — Vérifier que les bons logiciels sont installés

Pourquoi : ce projet a besoin de Python (pour le code) et PostgreSQL (pour la base de données). Autant vérifier maintenant que se battre avec une erreur cryptique plus tard.



powershell
python --version

Il faut Python **3.11 ou supérieur**. Si `python` ne fonctionne pas sous Windows, essaie :



```
powershell  
py --version
```

Ensuite, PostgreSQL et ses outils en ligne de commande :



```
powershell  
psql --version  
createdb --version  
pg_isready --host=localhost --port=5432
```

Résultat attendu : PostgreSQL 15 ou supérieur, et le message `accepting connections` pour `pg_isready` (ça veut dire que le serveur PostgreSQL tourne et accepte des connexions).

Si `psql` est introuvable sous Windows, ajoute ce dossier au `PATH` (en adaptant le numéro de version installée) :



```
text  
C:\Program Files\PostgreSQL\15\bin
```

Puis ferme et rouvre PowerShell — le changement de `PATH` n'est pris en compte qu'au redémarrage du terminal.

Point de contrôle : ne continue que si `python --version` et `psql --version` répondent sans erreur.

Étape 3 — Créer un environnement Python isolé

Pourquoi : sans ça, les bibliothèques de ce projet s'installeraient dans le Python "global" de ton ordinateur, au risque d'entrer en conflit avec d'autres projets. Un environnement virtuel (`.venv`) est un dossier isolé, propre à ce projet.

Sous Windows PowerShell :



```
powershell  
py -3.11 -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip
```

Sous Linux ou macOS :



```
bash
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Résultat attendu : le terminal affiche `(.venv)` au début de la ligne. Tu peux vérifier quel Python est réellement actif :



```
powershell
python -c "import sys; print(sys.executable)"
```

Le chemin doit contenir `simulateur_V3` puis `.venv`.

Étape 4 — Créer les dossiers et fichiers vides

Pourquoi : on prépare tous les emplacements de fichiers à l'avance, pour ensuite les remplir un par un sans se tromper de nom.

Sous Windows PowerShell :



```
powershell
New-Item -ItemType Directory -Force app/models, alembic/versions, docs
New-Item -ItemType File -Force requirements.txt, alembic.ini, setup_db.sh
New-Item -ItemType File -Force app/__init__.py, app/database.py
New-Item -ItemType File -Force app/models/__init__.py, app/models/base.py, app/models/schema.py
New-Item -ItemType File -Force alembic/env.py, alembic/script.py.mako
New-Item -ItemType File -Force alembic/versions/__init__.py
New-Item -ItemType File -Force alembic/versions/20260814_0001_schema_initial.py
New-Item -ItemType File -Force docs/schema_er.md
```

Sous Linux ou macOS :



```
bash
mkdir -p app/models alembic/versions docs
touch requirements.txt alembic.ini setup_db.sh
touch app/__init__.py app/database.py
touch app/models/__init__.py app/models/base.py app/models/schema.py
touch alembic/env.py alembic/script.py.mako
touch alembic/versions/__init__.py
touch alembic/versions/20260814_0001_schema_initial.py
touch docs/schema_er.md
```

Important : ne lance surtout pas `alembic init alembic`. Cette commande générerait des fichiers Alembic génériques qui écraseraient la configuration déjà prête (adaptée au mode asynchrone) fournie section 5.

Point de contrôle sous PowerShell :



powershell

`Get-ChildItem -Recurse -File | Select-Object FullName`

Compare la sortie avec l'arborescence de la section 4 du document original (même liste de fichiers).

Étape 5 — Copier chaque bloc de code dans le bon fichier

Pourquoi : la section 5 de ce document contient le contenu exact de chaque fichier créé à l'étape précédente. Il suffit maintenant de copier chaque bloc dans le fichier correspondant.

Ouvrir le dossier dans Visual Studio Code :



powershell

`code .`

Si la commande `code` n'existe pas, ouvre VS Code manuellement puis **Fichier > Ouvrir un dossier**, et sélectionne `simulateur_V3`.

Correspondance fichier ↔ section de ce document :

Section	Fichier à remplir
---------	-------------------

4.1	<code>requirements.txt</code>
-----	-------------------------------

4.2	<code>app/__init__.py</code>
-----	------------------------------

4.3	<code>app/database.py</code>
-----	------------------------------

4.4	<code>app/models/base.py</code>
-----	---------------------------------

4.5	<code>app/models/schema.py</code>
-----	-----------------------------------

4.6	<code>app/models/__init__.py</code>
-----	-------------------------------------

4.7	<code>alembic.ini</code>
-----	--------------------------

4.8	<code>alembic/env.py</code>
-----	-----------------------------

4.9	<code>alembic/script.py.mako</code>
-----	-------------------------------------

4.10	<code>alembic/versions/__init__.py</code>
------	---

4.11	<code>alembic/versions/20260814_0001_schema_initial.py</code>
------	---

4.12	<code>setup_db.sh</code>
------	--------------------------

Pour `docs/schema_er.md`, copie d'abord cet en-tête :



markdown

<!-- Diagramme entité-relation du schéma PostgreSQL CMU. -->

Diagramme entité-relation

puis colle immédiatement après le bloc Mermaid complet de la section 5 (garde bien les lignes d'ouverture et de fermeture avec les trois accents graves ```).

Méthode sûre : remplis et enregistre un fichier à la fois, et vérifie le nom/l'extension avant de passer au suivant — une faute de frappe dans un nom de fichier Python cassera les imports plus tard.

Étape 6 — Installer les dépendances du projet

Pourquoi : les fichiers Python que tu viens de copier utilisent des bibliothèques externes (SQLAlchemy, Alembic...) listées dans `requirements.txt`. Il faut les installer dans l'environnement virtuel.



```
powershell  
python -m pip install -r requirements.txt
```

Résultat attendu : aucune erreur, et les bibliothèques listées apparaissent avec `pip list`.

Étape 7 — Vérifier le code avant de toucher PostgreSQL

Pourquoi : autant détecter une faute de frappe ou un import cassé **avant** de lancer une migration, plutôt qu'après (où l'erreur serait plus difficile à situer).



```
powershell  
python -m compileall app alembic
```

Résultat attendu : aucune erreur affichée (juste des lignes `Compiling...` qui se terminent normalement).

Étape 8 — Préparer les paramètres de connexion PostgreSQL

Pourquoi : Alembic a besoin de savoir où se connecter (hôte, port, utilisateur, mot de passe, nom de la base). Adapte les valeurs suivantes à ton installation PostgreSQL locale avant de continuer (section 8 du guide original pour le détail complet).

Étape 9 — Créer la base et appliquer la migration

C'est ici que les 22 tables sont réellement créées dans PostgreSQL.

Avec Git Bash, Linux ou macOS



```
bash
```

```
chmod +x setup_db.sh
./setup_db.sh
```

Avec PowerShell (sans Bash)

Crée la base manuellement (pgAdmin ou `createdb`), puis :



```
powershell
$env:DATABASE_URL = "postgresql+asyncpg://postgres:postgres@localhost:5432/cmu_simulator"
python -m alembic upgrade head
```

Ce qui se passe concrètement : Alembic lit la migration `20260814_0001_schema_initial.py`, qui contient les instructions "crée telle table avec telles colonnes" pour les 22 tables, et les exécute en une seule transaction sur ta base PostgreSQL.

Étape 10 — Vérifier que l'installation est vraiment terminée

Pourquoi : une commande qui ne renvoie pas d'erreur ne garantit pas toujours que *tout* s'est bien passé — mieux vaut vérifier concrètement.



```
bash
python -m alembic current
```

Résultat attendu : `20260814_0001 (head)` — ça confirme que la base est bien à la dernière révision connue.

Puis (dans `psql` ou pgAdmin) :



```
sql
SELECT COUNT(*) AS nombre_tables
FROM information_schema.tables
WHERE table_schema = 'public'
AND table_name LIKE 'TB_%';
```

Résultat attendu : `22`.

Étape 11 — Garder une preuve pour ton mémoire

Génère le SQL généré par la migration, sans toucher la base, pour l'annexer à ton mémoire :



bash

```
python -m alembic upgrade head --sql > schema_initial.sql
```

Checklist de fin d'étape

- ☐ `python --version` $\geq$ 3.11 et `psql --version` $\geq$ 15 ;
- ☐ `.venv` actif, dépendances installées ;
- ☐ tous les fichiers de l'arborescence créés et remplis ;
- ☐ `python -m compileall app alembic` sans erreur ;
- ☐ `python -m alembic current` $\rightarrow$ `20260814_0001 (head)` ;
- ☐ la requête `COUNT(*)` renvoie `22` ;
- ☐ `docs/schema_er.md` contient le diagramme Mermaid.

4. Le code complet, fichier par fichier, expliqué

Avant de copier chaque bloc, voici **ce que fait chaque fichier et pourquoi il existe** — c'est ce qui manquait dans le guide original pour bien comprendre (pas seulement exécuter).

Fichier**Rôle en une phrase**`requirements.txt`

La liste des bibliothèques Python nécessaires à cette étape (aucun code, juste une liste de dépendances).

`app/database.py`

Configure *comment* le code Python se connecte à PostgreSQL (l'équivalent d'un "branchement" réutilisable partout ailleurs dans le projet).

`app/models/base.py`

Définit le "moule" commun à toutes les tables : la classe `Base` (dont hérite chaque table) et `AuditMixin` (les 4 colonnes d'audit ajoutées automatiquement partout).

`app/models/schema.py`

Le cœur de l'étape : décrit les 22 tables comme des classes Python, avec leurs colonnes, clés et relations.

`alembic.ini` + `alembic/env.py` + `alembic/script.py.mako`

La configuration d'Alembic : où trouver la base, comment lire les modèles, quel modèle utiliser pour créer une nouvelle migration plus tard.

`alembic/versions/20260814_0001_schema_initial.py`

La migration elle-même : "crée toutes ces tables" (et son inverse, "supprime-les", si besoin d'annuler).

`setup_db.sh`

Un script qui automatise "crée la base PostgreSQL" + "applique la migration" en une seule commande.

4.1 requirements.txt

Chaque ligne est une bibliothèque Python et sa version exacte (pour que le projet fonctionne pareil sur toutes les machines) :

`SQLAlchemy` : l'ORM (voir section 0).

`alembic` : l'outil de migrations.

`asyncpg` et `psycopg` : les deux "pilotes" qui permettent à Python de parler concrètement au serveur PostgreSQL (l'un en mode asynchrone, l'autre utilisé par certains outils en ligne de commande d'Alembic).



```

text
```text
Dépendances de la couche de persistance et des migrations.
SQLAlchemy[asyncio]==2.0.43
alembic==1.16.4
asyncpg==0.30.0
psycopg[binary]==3.2.9
```

```

4.2 `app/__init__.py`

Un fichier presque vide : sa seule utilité est de dire à Python "le dossier `app/` est un paquet importable" (une convention Python classique, rien de plus).

```

python
```python
"""Initialise le paquet applicatif du simulateur CMU."""
```

```

4.3 `app/database.py`

Ce fichier centralise **la connexion** à PostgreSQL, pour que tout le reste du projet (plus tard, l'API) s'y connecte de la même façon sans dupliquer le code. Trois éléments clés :

`DATABASE_URL` : l'adresse de connexion. Elle a une valeur par défaut (utile en développement local) mais peut être remplacée par une variable d'environnement — utile pour ne jamais écrire de mot de passe en dur dans le code source.

`create_async_engine(..., pool_pre_ping=True)` : crée le "moteur" de connexion. `pool_pre_ping` vérifie qu'une connexion réutilisée est toujours valide avant de s'en servir (évite une erreur si PostgreSQL a coupé une connexion inactive depuis trop longtemps).

`get_database_session()` : une fonction qui fournit une session (une "conversation" avec la base) et garantit qu'elle se referme correctement après usage, même en cas d'erreur.

```

python

```

```
```python
"""Configure le moteur SQLAlchemy asynchrone et les sessions PostgreSQL."""

import os
from collections.abc import AsyncIterator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

La valeur locale reste surchargeable pour les autres environnements.
DATABASE_URL = os.getenv(
 "DATABASE_URL",
 "postgresql+asyncpg://postgres:postgres@localhost:5432/cmu_simulator",
)

Le pré-ping empêche la réutilisation d'une connexion PostgreSQL périmée.
engine = create_async_engine(DATABASE_URL, pool_pre_ping=True)
async_session_factory = async_sessionmaker(engine, expire_on_commit=False)

async def get_database_session() -> AsyncIterator[AsyncSession]:
 """Fournit une session asynchrone et garantit sa fermeture."""

 async with async_session_factory() as session:
 yield session
```
```

4.4 app/models/base.py

Deux éléments réutilisés par les 22 tables :

Base : la classe dont **toutes** les tables héritent — c'est ce qui permet à SQLAlchemy de savoir "quelles classes Python correspondent à des tables".

`AuditMixin`: un "mixin" (un bloc de colonnes réutilisable) qui ajoute automatiquement les 4 colonnes d'audit à n'importe quelle table qui l'hérite, sans avoir à les réécrire 22 fois. `server_default=func.now()` veut dire que PostgreSQL lui-même remplit `DATE_CREATION` avec l'heure actuelle si le code Python ne le fait pas explicitement.



python

```
```python
"""Définit la base déclarative et les colonnes d'audit communes."""

from datetime import datetime

from sqlalchemy import DateTime, String, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
 """Base déclarative commune à tous les modèles SQLAlchemy."""

class AuditMixin:
 """Ajoute les quatre colonnes d'audit obligatoires à une table."""

 # La valeur serveur couvre aussi les insertions effectuées hors de l'API.
 date_creation: Mapped[datetime] = mapped_column(
 "DATE_CREATION",
 DateTime(timezone=True),
 server_default=func.now(),
 nullable=False,
)
 utilisateur_id_creation: Mapped[str | None] = mapped_column(
 "UTILISATEUR_ID_CREATION", String(100)
)
 date_modification: Mapped[datetime | None] = mapped_column(
 "DATE_MODIFICATION", DateTime(timezone=True)
)
 utilisateur_id_modification: Mapped[str | None] = mapped_column(
```

```

 "UTILISATEUR_ID_MODIFICATION", String(100)
)
...

```

#### 4.5 `app/models/schema.py` — le fichier le plus important de cette étape

C'est ici que les 22 tables sont décrites. Le fichier est long (781 lignes) car il y a 22 tables, mais **elles suivent toutes le même schéma** — une fois que tu as compris une table, tu as compris les 22. Regardons un exemple concret extrait du fichier, la table des assurés :




```

python
class InsuredPerson(AuditMixin, Base):
 # Représente un assuré du référentiel CMU.

 __tablename__ = "TB_REF_ASSURES"

 personne_uuid: Mapped[UUID] = mapped_column(
 "PERSONNE_UUID", PostgreSQLUUID(as_uuid=True), primary_key=True
)
 numero_secu: Mapped[str] = mapped_column(
 "NUMERO_SECU", String(50), nullable=False, unique=True
)
 assure_nom: Mapped[str] = mapped_column("ASSURE_NOM", String(150), nullable=False)

 invoices: Mapped[list[Invoice]] = relationship(back_populates="insured_person")

```

Ligne par ligne :

`class InsuredPerson(AuditMixin, Base)` : cette classe Python représente une table. Elle hérite de `AuditMixin` (donc elle aura automatiquement les 4 colonnes d'audit) et de `Base` (donc SQLAlchemy la reconnaît comme une table).

`__tablename__ = "TB_REF_ASSURES"` : le vrai nom de la table dans PostgreSQL.

`personne_uuid: Mapped[UUID] = mapped_column("PERSONNE_UUID", ..., primary_key=True)` : une colonne nommée `PERSONNE_UUID` côté PostgreSQL (`personne_uuid` côté Python), de type UUID, et c'est la **clé primaire** de la table.

`nullable=False, unique=True`: cette colonne est obligatoire et ne peut pas être dupliquée entre deux assurés.

`invoices: Mapped[list[Invoice]] = relationship(...)`: ceci n'est **pas** une colonne SQL — c'est une commodité Python qui permet d'écrire `mon_assure.invoices` pour récupérer directement la liste de ses factures, sans écrire de requête SQL à la main. La vraie clé étrangère est définie du **côté de la table** `Invoice` (voir plus bas dans le fichier).

Tu retrouveras ce même schéma pour les 21 autres tables : une classe, un `__tablename__`, des colonnes avec `mapped_column`, et des `relationship` pour naviguer facilement entre tables liées. Certaines tables ont une clé **composite** (plusieurs colonnes ensemble comme clé primaire) : tu les reconnaîtras à un `PrimaryKeyConstraint` ou plusieurs colonnes avec `primary_key=True`.

Voici le fichier complet à copier dans `app/models/schema.py`:



python

```
```python
"""Décrit toutes les tables TB_XXX et leurs relations explicites."""

from __future__ import annotations

from datetime import date
from decimal import Decimal
from uuid import UUID

from sqlalchemy import (
    Boolean,
    CheckConstraint,
    Date,
    ForeignKey,
    ForeignKeyConstraint,
    Integer,
    Numeric,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.dialects.postgresql import UUID as PostgreSQLUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import AuditMixin, Base

class InsuredPerson(AuditMixin, Base):
    """Représente un assuré du référentiel CMU."""
```

```
__tablename__ = "TB_REF_ASSURES"
```

```
personne_uuid: Mapped[UUID] = mapped_column(
    "PERSONNE_UUID", PostgreSQLUUID(as_uuid=True), primary_key=True
)
numero_recepisse: Mapped[str | None] = mapped_column("NUMERO_RECEPISSE", String(50))
assure_numero_identifiant: Mapped[str | None] = mapped_column(
    "ASSURE_NUMERO_IDENTIFIANT", String(50)
)
numero_secu: Mapped[str] = mapped_column(
    "NUMERO_SECU", String(50), nullable=False, unique=True
)
civillite_code: Mapped[str | None] = mapped_column("CIVILITE_CODE", String(10))
assure_nom: Mapped[str] = mapped_column("ASSURE_NOM", String(150), nullable=False)
assure_nom_patronymique: Mapped[str | None] = mapped_column(
    "ASSURE_NOM_PATRONYMIQUE", String(150)
)

invoices: Mapped[list[Invoice]] = relationship(back_populates="insured_person")
prior_authorizations: Mapped[list[PriorAuthorization]] = relationship(
    back_populates="insured_person"
)
```

```
class HealthCenter(AuditMixin, Base):
    """Décrit un centre de santé agréé par la CMU."""
```

```
__tablename__ = "TB_REF_CENTRES_SANTE"
```

```
centre_sante_code: Mapped[str] = mapped_column(
```

```
    "CENTRE_SANTE_CODE", String(30), primary_key=True
)
collectivite_code: Mapped[str | None] = mapped_column("COLLECTIVITE_CODE", String(30))
type_etablissement_sanitaire_code: Mapped[str | None] = mapped_column(
    "TYPE_ETABLISSEMENT_SANITAIRE_CODE", String(30)
)
centre_sante_numero_immatriculation: Mapped[str] = mapped_column(
    "CENTRE_SANTE_NUMERO_IMMATRICULATION", String(50), nullable=False, unique=True
)
centre_sante_denomination: Mapped[str] = mapped_column(
    "CENTRE_SANTE_DENOMINATION", String(255), nullable=False
)

agent_assignments: Mapped[list[CenterHealthAgent]] = relationship(
    back_populates="health_center", cascade="all, delete-orphan"
)
professional_assignments: Mapped[list[HealthProfessionalCenter]] = relationship(
    back_populates="health_center", cascade="all, delete-orphan"
)
invoices: Mapped[list[Invoice]] = relationship(back_populates="health_center")
prior_authorizations: Mapped[list[PriorAuthorization]] = relationship(
    back_populates="health_center"
)
```

```
class Agent(AuditMixin, Base):
    """Représente un agent CMU et son rôle opérationnel."""

    __tablename__ = "TB_REF_AGENTS"
    __table_args__ = (
```

```
CheckConstraint(
    "AGENT_TYPE_CODE" IN (\'accueil\', \'medecin_conseil\', \'autre\'),
    name="CK_TB_REF_AGENTS_TYPE",
),
)

agent_code: Mapped[str] = mapped_column("AGENT_CODE", String(30), primary_key=True)
agent_code_gestion: Mapped[str | None] = mapped_column("AGENT_CODE_GESTION", String(30))
agent_prenoms: Mapped[str] = mapped_column("AGENT_PRENOMS", String(150), nullable=False)
agent_nom: Mapped[str] = mapped_column("AGENT_NOM", String(150), nullable=False)
agent_email: Mapped[str] = mapped_column(
    "AGENT_EMAIL", String(255), nullable=False, unique=True
)
agent_type_code: Mapped[str] = mapped_column(
    "AGENT_TYPE_CODE", String(30), nullable=False
)

center_assignments: Mapped[list[CenterHealthAgent]] = relationship(
    back_populates="agent", cascade="all, delete-orphan"
)
prior_authorization_statuses: Mapped[list[PriorAuthorizationStatus]] = relationship(
    back_populates="agent"
)

class CenterHealthAgent(AuditMixin, Base):
    """Historise l'affectation d'un agent à un centre de santé."""

    __tablename__ = "TB_CENTRES_SANTE_AGENTS"
```

```
centre_sante_code: Mapped[str] = mapped_column(
    "CENTRE_SANTE_CODE",
    ForeignKey("TB_REF_CENTRES_SANTE.CENTRE_SANTE_CODE"),
    primary_key=True,
)
agent_code: Mapped[str] = mapped_column(
    "AGENT_CODE", ForeignKey("TB_REF_AGENTS.AGENT_CODE"), primary_key=True
)
date_debut: Mapped[date] = mapped_column("DATE_DEBUT", Date, primary_key=True)
date_fin: Mapped[date | None] = mapped_column("DATE_FIN", Date)

health_center: Mapped[HealthCenter] = relationship(back_populates="agent_assignments")
agent: Mapped[Agent] = relationship(back_populates="center_assignments")
```

```
class HealthProfessional(AuditMixin, Base):
```

```
    """Décrit un professionnel intervenant dans le parcours de soins."""
```

```
    __tablename__ = "TB_REF_PROFESSIONNELS_SANTE"
```

```
    professionnel_sante_code: Mapped[str] = mapped_column(
        "PROFESSIONNEL_SANTE_CODE", String(30), primary_key=True
    )
```

```
    nom: Mapped[str] = mapped_column("NOM", String(150), nullable=False)
```

```
    prenoms: Mapped[str] = mapped_column("PRENOMS", String(150), nullable=False)
```

```
    type_code: Mapped[str] = mapped_column("TYPE_CODE", String(30), nullable=False)
```

```
    numero_ordre: Mapped[str | None] = mapped_column("NUMERO_ORDRE", String(50))
```

```
    statut: Mapped[str] = mapped_column("STATUT", String(30), nullable=False)
```

```
    specialty_assignments: Mapped[list[HealthProfessionalMedicalSpecialty]] = relationship(
```

```
    back_populates="health_professional", cascade="all, delete-orphan"
)
center_assignments: Mapped[list[HealthProfessionalCenter]] = relationship(
    back_populates="health_professional", cascade="all, delete-orphan"
)
invoice_provisions: Mapped[list[InvoiceProvision]] = relationship(
    back_populates="health_professional"
)
prior_authorization_medical_acts: Mapped[list[PriorAuthorizationMedicalAct]] = relationship(
    back_populates="health_professional"
)
prior_authorization_provisions: Mapped[list[PriorAuthorizationProvision]] = relationship(
    back_populates="health_professional"
)
```

```
class MedicalSpecialty(AuditMixin, Base):
```

```
    """Référence une spécialité médicale reconnue."""
```

```
    __tablename__ = "TB_REF_SPECIALITES_MEDICALES"
```

```
    specialite_medicale_code: Mapped[str] = mapped_column(
        "SPECIALITE_MEDICALE_CODE", String(30), primary_key=True
    )
```

```
    denomination: Mapped[str] = mapped_column("DENOMINATION", String(255), nullable=False)
```

```
    professional_assignments: Mapped[list[HealthProfessionalMedicalSpecialty]] = relationship(
        back_populates="medical_specialty", cascade="all, delete-orphan"
    )
```

```
class HealthProfessionalMedicalSpecialty(AuditMixin, Base):
    """Historise les spécialités d'un professionnel de santé."""

    __tablename__ = "TB_REF_PROFESSIONNELS_SANTE_SPECIALITES_MEDICALES"

    professionnel_sante_code: Mapped[str] = mapped_column(
        "PROFESSIONNEL_SANTE_CODE",
        ForeignKey("TB_REF_PROFESSIONNELS_SANTE.PROFESSIONNEL_SANTE_CODE"),
        primary_key=True,
    )
    specialite_medicale_code: Mapped[str] = mapped_column(
        "SPECIALITE_MEDICALE_CODE",
        ForeignKey("TB_REF_SPECIALITES_MEDICALES.SPECIALITE_MEDICALE_CODE"),
        primary_key=True,
    )
    date_debut: Mapped[date] = mapped_column("DATE_DEBUT", Date, primary_key=True)
    date_fin: Mapped[date | None] = mapped_column("DATE_FIN", Date)

    health_professional: Mapped[HealthProfessional] = relationship(
        back_populates="specialty_assignments"
    )
    medical_specialty: Mapped[MedicalSpecialty] = relationship(
        back_populates="professional_assignments"
    )

class HealthProfessionalCenter(AuditMixin, Base):
    """Historise l'exercice d'un professionnel dans un centre."""
```

```
__tablename__ = "TB_PROFESSIONNELS_SANTE_CENTRES_SANTE"

professionnel_sante_code: Mapped[str] = mapped_column(
    "PROFESSIONNEL_SANTE_CODE",
    ForeignKey("TB_REF_PROFESSIONNELS_SANTE.PROFESSIONNEL_SANTE_CODE"),
    primary_key=True,
)
centre_sante_code: Mapped[str] = mapped_column(
    "CENTRE_SANTE_CODE",
    ForeignKey("TB_REF_CENTRES_SANTE.CENTRE_SANTE_CODE"),
    primary_key=True,
)
date_debut: Mapped[date] = mapped_column("DATE_DEBUT", Date, primary_key=True)
date_fin: Mapped[date | None] = mapped_column("DATE_FIN", Date)

health_professional: Mapped[HealthProfessional] = relationship(
    back_populates="center_assignments"
)
health_center: Mapped[HealthCenter] = relationship(
    back_populates="professional_assignments"
)

class Pathology(AuditMixin, Base):
    """Référence une version datée d'une pathologie interne."""

    __tablename__ = "TB_REF_PATHOLOGIES"

    pathologie_code: Mapped[str] = mapped_column(
        "PATHOLOGIE_CODE", String(3), primary_key=True
```

```
)  
pathologie_date_debut: Mapped[date] = mapped_column(  
    "PATHOLOGIE_DATE_DEBUT", Date, primary_key=True  
)  
sous_chapitre_code: Mapped[str | None] = mapped_column("SOUS_CHAPITRE_CODE", String(30))  
pathologie_denomination: Mapped[str] = mapped_column(  
    "PATHOLOGIE_DENOMINATION", String(255), nullable=False  
)  
pathologie_date_fin: Mapped[date | None] = mapped_column("PATHOLOGIE_DATE_FIN", Date)  
pathologie_statut: Mapped[str] = mapped_column(  
    "PATHOLOGIE_STATUT", String(30), nullable=False  
)  
  
invoice_pathologies: Mapped[list[InvoicePathology]] = relationship(  
    back_populates="pathology"  
)
```

```
class Medication(AuditMixin, Base):  
    """Référence une version datée d'un médicament."""
```

```
    __tablename__ = "TB_REF_MEDICAMENTS"
```

```
    medicament_code: Mapped[str] = mapped_column(  
        "MEDICAMENT_CODE", String(30), primary_key=True  
    )  
    medicament_date_debut: Mapped[date] = mapped_column(  
        "MEDICAMENT_DATE_DEBUT", Date, primary_key=True  
    )  
    type_code: Mapped[str | None] = mapped_column("TYPE_CODE", String(30))
```

```
medicament_denomination: Mapped[str] = mapped_column(
    "MEDICAMENT_DENOMINATION", String(255), nullable=False
)
medicament_ean13: Mapped[str | None] = mapped_column("MEDICAMENT_EAN13", String(13))
dci_code: Mapped[str | None] = mapped_column("DCI_CODE", String(30))
laboratoire_code: Mapped[str | None] = mapped_column("LABORATOIRE_CODE", String(30))
famille_forme_code: Mapped[str | None] = mapped_column("FAMILLE_FORME_CODE", String(30))
conditionnement_code: Mapped[str | None] = mapped_column("CONDITIONNEMENT_CODE", String(30))
presentation_code: Mapped[str | None] = mapped_column("PRESENTATION_CODE", String(30))
liste_types_factures: Mapped[str | None] = mapped_column("LISTE_TYPES_FACTURES", Text)
liste_genres: Mapped[str | None] = mapped_column("LISTE_GENRES", Text)
medicament_quantite_maximum: Mapped[int | None] = mapped_column(
    "MEDICAMENT_QUANTITE_MAXIMUM", Integer
)
medicament_age_minimum: Mapped[int | None] = mapped_column(
    "MEDICAMENT_AGE_MINIMUM", Integer
)
medicament_age_maximum: Mapped[int | None] = mapped_column(
    "MEDICAMENT_AGE_MAXIMUM", Integer
)
medicament_hapax_statut: Mapped[bool] = mapped_column(
    "MEDICAMENT_HAPAX_STATUT", Boolean, nullable=False, default=False
)
medicament_tarif_default: Mapped[Decimal] = mapped_column(
    "MEDICAMENT_TARIF_DEFAULT", Numeric(15, 2), nullable=False
)
medicament_statut: Mapped[str] = mapped_column(
    "MEDICAMENT_STATUT", String(30), nullable=False
)
medicament_date_fin: Mapped[date | None] = mapped_column("MEDICAMENT_DATE_FIN", Date)
```

```
class MedicalAct(AuditMixin, Base):
    """Référence une version datée d'un acte médical."""

    __tablename__ = "TB_REF_ACTES_MEDICAUX"

    acte_medical_code: Mapped[str] = mapped_column(
        "ACTE_MEDICAL_CODE", String(30), primary_key=True
    )
    acte_medical_date_debut: Mapped[date] = mapped_column(
        "ACTE_MEDICAL_DATE_DEBUT", Date, primary_key=True
    )
    article_code: Mapped[str | None] = mapped_column("ARTICLE_CODE", String(30))
    acte_medical_type: Mapped[str] = mapped_column(
        "ACTE_MEDICAL_TYPE", String(30), nullable=False
    )
    acte_medical_denomination: Mapped[str] = mapped_column(
        "ACTE_MEDICAL_DENOMINATION", String(255), nullable=False
    )
    liste_types_factures: Mapped[str | None] = mapped_column("LISTE_TYPES_FACTURES", Text)
    liste_genres: Mapped[str | None] = mapped_column("LISTE_GENRES", Text)
    acte_medical_age_minimum: Mapped[int | None] = mapped_column(
        "ACTE_MEDICAL_AGE_MINIMUM", Integer
    )
    acte_medical_age_maximum: Mapped[int | None] = mapped_column(
        "ACTE_MEDICAL_AGE_MAXIMUM", Integer
    )
    acte_medical_quantite_maximum: Mapped[int | None] = mapped_column(
        "ACTE_MEDICAL_QUANTITE_MAXIMUM", Integer
    )
```

```
)
acte_medical_hapax_statut: Mapped[bool] = mapped_column(
    "ACTE_MEDICAL_HAPAX_STATUT", Boolean, nullable=False, default=False
)
acte_medical_statut: Mapped[str] = mapped_column(
    "ACTE_MEDICAL_STATUT", String(30), nullable=False
)
acte_medical_date_fin: Mapped[date | None] = mapped_column(
    "ACTE_MEDICAL_DATE_FIN", Date
)
```

```
class TypeInvoice(AuditMixin, Base):
    """Référence une version datée d'un type de facture."""

    __tablename__ = "TB_TV_TYPES_FACTURES"

    type_facture_code: Mapped[str] = mapped_column(
        "TYPE_FACTURE_CODE", String(30), primary_key=True
    )
    type_facture_date_debut: Mapped[date] = mapped_column(
        "TYPE_FACTURE_DATE_DEBUT", Date, primary_key=True
    )
    type_facture_denomination: Mapped[str] = mapped_column(
        "TYPE_FACTURE_DENOMINATION", String(255), nullable=False
    )
    type_facture_date_fin: Mapped[date | None] = mapped_column(
        "TYPE_FACTURE_DATE_FIN", Date
    )
    type_facture_statut: Mapped[str] = mapped_column(
```

```
    "TYPE_FACTURE_STATUT", String(30), nullable=False
)

class Invoice(AuditMixin, Base):
    """Porte la facture centrale d'un parcours de soins."""

    __tablename__ = "TB_FACTURES"

    facture_numero: Mapped[str] = mapped_column(
        "FACTURE_NUMERO", String(50), primary_key=True
    )
    produit_code: Mapped[str | None] = mapped_column("PRODUIT_CODE", String(30))
    regime_code: Mapped[str | None] = mapped_column("REGIME_CODE", String(30))
    regime_taux: Mapped[Decimal | None] = mapped_column("REGIME_TAUX", Numeric(5, 2))
    organisme_code: Mapped[str | None] = mapped_column("ORGANISME_CODE", String(30))
    assurance_code: Mapped[str | None] = mapped_column("ASSURANCE_CODE", String(30))
    personne_uuid: Mapped[UUID] = mapped_column(
        "PERSONNE_UUID", ForeignKey("TB_REF_ASSURES.PERSONNE_UUID"), nullable=False
    )
    type_facture_code: Mapped[str] = mapped_column(
        "TYPE_FACTURE_CODE", String(30), nullable=False
    )
    facture_date_soins: Mapped[date] = mapped_column(
        "FACTURE_DATE_SOINS", Date, nullable=False
    )
    dossier_numero: Mapped[str] = mapped_column(
        "DOSSIER_NUMERO", String(50), nullable=False
    )
    centre_sante_code: Mapped[str] = mapped_column(
```

```
"CENTRE_SANTE_CODE",
ForeignKey("TB_REF_CENTRES_SANTE.CENTRE_SANTE_CODE"),
nullable=False,
)
centre_sante_type_code: Mapped[str | None] = mapped_column(
    "CENTRE_SANTE_TYPE_CODE", String(30)
)
centre_sante_type_libelle: Mapped[str | None] = mapped_column(
    "CENTRE_SANTE_TYPE_LIBELLE", String(255)
)
reseau_code: Mapped[str | None] = mapped_column("RESEAU_CODE", String(30))
# use_alter résout le cycle physique entre facture et entente préalable.
entente_prealable_id: Mapped[int | None] = mapped_column(
    "ENTENTE_PREALABLE_ID",
    ForeignKey(
        "TB_ENTENTES_PREALABLES.ENTENTE_PREALABLE_ID",
        name="FK_FACTURE_ENTENTE",
        use_alter=True,
    ),
)

insured_person: Mapped[InsuredPerson] = relationship(back_populates="invoices")
health_center: Mapped[HealthCenter] = relationship(back_populates="invoices")
prior_authorization: Mapped[PriorAuthorization | None] = relationship(
    foreign_keys=[entente_prealable_id], back_populates="linked_invoices"
)
linked_prior_authorizations: Mapped[list[PriorAuthorization]] = relationship(
    foreign_keys="PriorAuthorization.facture_numero", back_populates="invoice"
)
pathologies: Mapped[list[InvoicePathology]] = relationship(
```

```
    back_populates="invoice", cascade="all, delete-orphan"
)
prescriptions: Mapped[list[InvoicePrescription]] = relationship(
    back_populates="invoice", cascade="all, delete-orphan"
)
provisions: Mapped[list[InvoiceProvision]] = relationship(
    back_populates="invoice", cascade="all, delete-orphan"
)
rejections: Mapped[list[InvoiceRejection]] = relationship(
    back_populates="invoice", cascade="all, delete-orphan"
)
statuses: Mapped[list[InvoiceStatus]] = relationship(
    back_populates="invoice", cascade="all, delete-orphan"
)
```

```
class InvoicePathology(AuditMixin, Base):
    """Associe une facture à une version précise de pathologie."""

    __tablename__ = "TB_FACTURES_PATHOLOGIES"
    __table_args__ = (
        ForeignKeyConstraint(
            ["PATHOLOGIE_CODE", "PATHOLOGIE_DATE_DEBUT"],
            [
                "TB_REF_PATHOLOGIES.PATHOLOGIE_CODE",
                "TB_REF_PATHOLOGIES.PATHOLOGIE_DATE_DEBUT",
            ],
            name="FK_FACTURE_PATHOLOGIE_REF",
        ),
    )
```

```
facture_numero: Mapped[str] = mapped_column(
    "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO"), primary_key=True
)
pathologie_code: Mapped[str] = mapped_column(
    "PATHOLOGIE_CODE", String(3), primary_key=True
)
pathologie_date_debut: Mapped[date] = mapped_column(
    "PATHOLOGIE_DATE_DEBUT", Date, primary_key=True
)
pathologie_observations: Mapped[str | None] = mapped_column(
    "PATHOLOGIE_OBSERVATIONS", Text
)
pathologie_date_fin: Mapped[date | None] = mapped_column(
    "PATHOLOGIE_DATE_FIN", Date
)

invoice: Mapped[Invoice] = relationship(back_populates="pathologies")
pathology: Mapped[Pathology] = relationship(back_populates="invoice_pathologies")
```

```
class InvoicePrescription(AuditMixin, Base):
    """Enregistre une prescription liée à une facture."""

    __tablename__ = "TB_FACTURES_PRESCRIPTIONS"

    facture_numero: Mapped[str] = mapped_column(
        "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO"), primary_key=True
    )
    prescription_code: Mapped[str] = mapped_column(
```

```
    "PRESCRIPTION_CODE", String(30), primary_key=True
)
date_debut: Mapped[date] = mapped_column("DATE_DEBUT", Date, primary_key=True)
prescription_quantite: Mapped[Decimal | None] = mapped_column(
    "PRESCRIPTION_QUANTITE", Numeric(10, 2)
)
prescription_posologie: Mapped[str | None] = mapped_column(
    "PRESCRIPTION_POSOLOGIE", Text
)
prescription_duree: Mapped[int | None] = mapped_column("PRESCRIPTION_DUREE", Integer)
prescription_renseignements_cliniques: Mapped[str | None] = mapped_column(
    "PRESCRIPTION_RENSEIGNEMENTS_CLINIQUES", Text
)
date_fin: Mapped[date | None] = mapped_column("DATE_FIN", Date)

invoice: Mapped[Invoice] = relationship(back_populates="prescriptions")
```

```
class InvoiceProvision(AuditMixin, Base):
```

```
    """Détaille une prestation et son calcul de remboursement."""
```

```
    __tablename__ = "TB_FACTURES_PRESTATIONS"
```

```
    facture_numero: Mapped[str] = mapped_column(
        "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO"), primary_key=True
    )
    prestation_code: Mapped[str] = mapped_column(
        "PRESTATION_CODE", String(30), primary_key=True
    )
    professionnel_sante_code: Mapped[str] = mapped_column(
```

```
"PROFESSIONNEL_SANTE_CODE",
ForeignKey("TB_REF_PROFESSIONNELS_SANTE.PROFESSIONNEL_SANTE_CODE"),
nullable=False,
)
statut_remboursement: Mapped[str | None] = mapped_column(
    "STATUT_REMBOURSEMENT", String(30)
)
motif_non_remboursement: Mapped[str | None] = mapped_column(
    "MOTIF_NON_REMBOURSEMENT", Text
)
prestation_base_remboursement: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_BASE_REMBOURSEMENT", Numeric(15, 2)
)
prestation_taux_remboursement: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_TAUX_REMBOURSEMENT", Numeric(5, 2)
)
prestation_quantite_prescrire: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_QUANTITE_PRESCRIRE", Numeric(10, 2)
)
prestation_quantite_servie: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_QUANTITE_SERVIE", Numeric(10, 2)
)
prestation_prix_unitaire: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_PRIX_UNITAIRE", Numeric(15, 2)
)
prestation_montant_depense: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_MONTANT_DEPENSE", Numeric(15, 2)
)
prestation_montant_rq: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_MONTANT_RQ", Numeric(15, 2)
```

```
)
prestation_montant_complementaire: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_MONTANT_COMPLEMENTAIRE", Numeric(15, 2)
)
prestation_montant_remise: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_MONTANT_REMISE", Numeric(15, 2)
)
prestation_montant_assure: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_MONTANT_ASSURE", Numeric(15, 2)
)
prestation_date_debut: Mapped[date | None] = mapped_column(
    "PRESTATION_DATE_DEBUT", Date
)
prestation_date_fin: Mapped[date | None] = mapped_column("PRESTATION_DATE_FIN", Date)
dents_numeros: Mapped[str | None] = mapped_column("DENTS_NUMEROS", String(100))
statut_code: Mapped[str | None] = mapped_column("STATUT_CODE", String(30))
motif_rejet_code: Mapped[str | None] = mapped_column("MOTIF_REJET_CODE", String(30))
prestation_observations: Mapped[str | None] = mapped_column(
    "PRESTATION_OBSERVATIONS", Text
)

invoice: Mapped[Invoice] = relationship(back_populates="provisions")
health_professional: Mapped[HealthProfessional] = relationship(
    back_populates="invoice_provisions"
)
```

```
class InvoiceRejection(AuditMixin, Base):
    """Historise les rejets appliqués à une facture."""
```

```
__tablename__ = "TB_FACTURES_REJETS"
```

```
facture_numero: Mapped[str] = mapped_column(
    "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO"), primary_key=True
)
rejet_code: Mapped[str] = mapped_column("REJET_CODE", String(30), primary_key=True)
rejet_date_debut: Mapped[date] = mapped_column(
    "REJET_DATE_DEBUT", Date, primary_key=True
)
rejet_date_fin: Mapped[date | None] = mapped_column("REJET_DATE_FIN", Date)

invoice: Mapped[Invoice] = relationship(back_populates="rejections")
```

```
class InvoiceStatus(AuditMixin, Base):
```

```
    """Historise les changements de statut d'une facture."""
```

```
__tablename__ = "TB_FACTURES_STATUTS"
```

```
facture_numero: Mapped[str] = mapped_column(
    "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO"), primary_key=True
)
statut_code: Mapped[str] = mapped_column("STATUT_CODE", String(30), primary_key=True)
statut_date_debut: Mapped[date] = mapped_column(
    "STATUT_DATE_DEBUT", Date, primary_key=True
)
statut_observations: Mapped[str | None] = mapped_column("STATUT_OBSERVATIONS", Text)
statut_date_fin: Mapped[date | None] = mapped_column("STATUT_DATE_FIN", Date)

invoice: Mapped[Invoice] = relationship(back_populates="statutes")
```

```
class PriorAuthorization(AuditMixin, Base):
    """Porte une demande d'entente préalable."""

    __tablename__ = "TB_ENTENTES_PREALABLES"
    __table_args__ = (
        UniqueConstraint("ENTENTE_PREALABLE_NUMERO", name="UQ_ENTENTE_NUMERO"),
    )

    entente_preable_id: Mapped[int] = mapped_column(
        "ENTENTE_PREALABLE_ID", Integer, primary_key=True, autoincrement=True
    )
    entente_preable_numero: Mapped[str] = mapped_column(
        "ENTENTE_PREALABLE_NUMERO", String(50), nullable=False
    )
    centre_sante_code: Mapped[str] = mapped_column(
        "CENTRE_SANTE_CODE",
        ForeignKey("TB_REF_CENTRES_SANTE.CENTRE_SANTE_CODE"),
        nullable=False,
    )
    personne_uuid: Mapped[UUID] = mapped_column(
        "PERSONNE_UUID", ForeignKey("TB_REF_ASSURES.PERSONNE_UUID"), nullable=False
    )
    dossier_numero: Mapped[str] = mapped_column(
        "DOSSIER_NUMERO", String(50), nullable=False
    )
    entente_preable_date_debut: Mapped[date] = mapped_column(
        "ENTENTE_PREALABLE_DATE_DEBUT", Date, nullable=False
    )
```

```
entente_prealable_date_fin: Mapped[date | None] = mapped_column(
    "ENTENTE_PREALABLE_DATE_FIN", Date
)
entente_prealable_numero_organisme: Mapped[str | None] = mapped_column(
    "ENTENTE_PREALABLE_NUMERO_ORGANISME", String(50)
)
organisme_code: Mapped[str | None] = mapped_column("ORGANISME_CODE", String(30))
facture_numero: Mapped[str | None] = mapped_column(
    "FACTURE_NUMERO", ForeignKey("TB_FACTURES.FACTURE_NUMERO")
)
type_demande_code: Mapped[str] = mapped_column(
    "TYPE_DEMANDE_CODE", String(30), nullable=False
)
type_hospitalisation_code: Mapped[str | None] = mapped_column(
    "TYPE_HOSPITALISATION_CODE", String(30)
)

health_center: Mapped[HealthCenter] = relationship(
    back_populates="prior_authorizations"
)
insured_person: Mapped[InsuredPerson] = relationship(
    back_populates="prior_authorizations"
)
invoice: Mapped[Invoice | None] = relationship(
    foreign_keys=[facture_numero], back_populates="linked_prior_authorizations"
)
linked_invoices: Mapped[list[Invoice]] = relationship(
    foreign_keys="Invoice.entente_prealable_id", back_populates="prior_authorization"
)
statuses: Mapped[list[PriorAuthorizationStatus]] = relationship(
```

```
    back_populates="prior_authorization", cascade="all, delete-orphan"
)
medical_acts: Mapped[list[PriorAuthorizationMedicalAct]] = relationship(
    back_populates="prior_authorization", cascade="all, delete-orphan"
)
provisions: Mapped[list[PriorAuthorizationProvision]] = relationship(
    back_populates="prior_authorization", cascade="all, delete-orphan"
)

class PriorAuthorizationStatus(AuditMixin, Base):
    """Historise le traitement d'une entente par un médecin conseil."""

    __tablename__ = "TB_ENTENTES_PREALABLES_STATUTS"

    entente_prealable_id: Mapped[int] = mapped_column(
        "ENTENTE_PREALABLE_ID",
        ForeignKey("TB_ENTENTES_PREALABLES.ENTENTE_PREALABLE_ID"),
        primary_key=True,
    )
    statut_code: Mapped[str] = mapped_column("STATUT_CODE", String(30), primary_key=True)
    statut_date_debut: Mapped[date] = mapped_column(
        "STATUT_DATE_DEBUT", Date, primary_key=True
    )
    agent_code: Mapped[str] = mapped_column(
        "AGENT_CODE", ForeignKey("TB_REF_AGENTS.AGENT_CODE"), nullable=False
    )
    statut_date_fin: Mapped[date | None] = mapped_column("STATUT_DATE_FIN", Date)

    prior_authorization: Mapped[PriorAuthorization] = relationship(
```

```
    back_populates="statuses"
)
agent: Mapped[Agent] = relationship(back_populates="prior_authorization_statuses")

class PriorAuthorizationMedicalAct(AuditMixin, Base):
    """Détaille la décision d'entente pour un acte médical."""

    __tablename__ = "TB_ENTENTES_PREALABLES_ACTES_MEDICAUX"

    entente_prealable_id: Mapped[int] = mapped_column(
        "ENTENTE_PREALABLE_ID",
        ForeignKey("TB_ENTENTES_PREALABLES.ENTENTE_PREALABLE_ID"),
        primary_key=True,
    )
    acte_medical_code: Mapped[str] = mapped_column(
        "ACTE_MEDICAL_CODE", String(30), primary_key=True
    )
    professionnel_sante_code: Mapped[str] = mapped_column(
        "PROFESSIONNEL_SANTE_CODE",
        ForeignKey("TB_REF_PROFESSIONNELS_SANTE.PROFESSIONNEL_SANTE_CODE"),
        nullable=False,
    )
    acte_medical_motif: Mapped[str | None] = mapped_column("ACTE_MEDICAL_MOTIF", Text)
    acte_medical_base_remboursement: Mapped[Decimal | None] = mapped_column(
        "ACTE_MEDICAL_BASE_REMBOURSEMENT", Numeric(15, 2)
    )
    acte_medical_taux_remboursement: Mapped[Decimal | None] = mapped_column(
        "ACTE_MEDICAL_TAUX_REMBOURSEMENT", Numeric(5, 2)
    )
```

```
acte_medical_montant_cmu: Mapped[Decimal | None] = mapped_column(
    "ACTE_MEDICAL_MONTANT_CMU", Numeric(15, 2)
)
acte_medical_montant_assure: Mapped[Decimal | None] = mapped_column(
    "ACTE_MEDICAL_MONTANT_ASSURE", Numeric(15, 2)
)
acte_medical_statut: Mapped[str] = mapped_column(
    "ACTE_MEDICAL_STATUT", String(30), nullable=False
)
acte_medical_motif_rejet: Mapped[str | None] = mapped_column(
    "ACTE_MEDICAL_MOTIF_REJET", Text
)

prior_authorization: Mapped[PriorAuthorization] = relationship(
    back_populates="medical_acts"
)
health_professional: Mapped[HealthProfessional] = relationship(
    back_populates="prior_authorization_medical_acts"
)

class PriorAuthorizationProvision(AuditMixin, Base):
    """Détaille la décision d'entente pour une prestation."""

    __tablename__ = "TB_ENTENTES_PREALABLES_PRESTATIONS"

    entente_prealable_id: Mapped[int] = mapped_column(
        "ENTENTE_PREALABLE_ID",
        ForeignKey("TB_ENTENTES_PREALABLES.ENTENTE_PREALABLE_ID"),
        primary_key=True,
```

```
)
prestation_code: Mapped[str] = mapped_column(
    "PRESTATION_CODE", String(30), primary_key=True
)
professionnel_sante_code: Mapped[str] = mapped_column(
    "PROFESSIONNEL_SANTE_CODE",
    ForeignKey("TB_REF_PROFESSIONNELS_SANTE.PROFESSIONNEL_SANTE_CODE"),
    nullable=False,
)
prestation_motif: Mapped[str | None] = mapped_column("PRESTATION_MOTIF", Text)
prestation_prix_unitaire: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_PRIX_UNITAIRE", Numeric(15, 2)
)
prestation_quantite: Mapped[Decimal | None] = mapped_column(
    "PRESTATION_QUANTITE", Numeric(10, 2)
)
prestation_statut: Mapped[str] = mapped_column(
    "PRESTATION_STATUT", String(30), nullable=False
)
prestation_motif_rejet: Mapped[str | None] = mapped_column(
    "PRESTATION_MOTIF_REJET", Text
)

prior_authorization: Mapped[PriorAuthorization] = relationship(
    back_populates="provisions"
)
health_professional: Mapped[HealthProfessional] = relationship(
    back_populates="prior_authorization_provisions"
)
...
```

4.6 `app/models/__init__.py`

Ce fichier réunit les 22 classes dans un seul endroit facile à importer (`from app.models import Base`), et garantit qu'Alembic "voit" bien toutes les tables avant de générer une migration.



python

```
```python
"""Expose tous les modèles pour la découverte complète des métadonnées."""

from app.models.base import Base
from app.models.schema import (
 Agent,
 CenterHealthAgent,
 HealthCenter,
 HealthProfessional,
 HealthProfessionalCenter,
 HealthProfessionalMedicalSpecialty,
 InsuredPerson,
 Invoice,
 InvoicePathology,
 InvoicePrescription,
 InvoiceProvision,
 InvoiceRejection,
 InvoiceStatus,
 MedicalAct,
 MedicalSpecialty,
 Medication,
 Pathology,
 PriorAuthorization,
 PriorAuthorizationMedicalAct,
 PriorAuthorizationProvision,
 PriorAuthorizationStatus,
 TypeInvoice,
)

__all__ = [name for name in globals() if not name.startswith("_")]
```

...

## 4.7 `alembic.ini`

Le fichier de configuration principal d'Alembic : où se trouve le dossier des migrations, quelle URL de base utiliser par défaut, etc.



ini

```
``ini
Configuration Alembic du schéma PostgreSQL CMU.
[alembic]
script_location = %(here)s/alembic
prepend_sys_path = .
path_separator = os
sqlalchemy.url = postgresql+asyncpg://postgres:postgres@localhost:5432/cmu_simulator

[loggers]
keys = root,sqlalchemy,alembic
[handlers]
keys = console
[formatters]
keys = generic
[logger_root]
level = WARN
handlers = console
qualname =
[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine
[logger_alembic]
level = INFO
handlers =
qualname = alembic
[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
```

```
formatter = generic
[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
'''
```

#### 4.8 alembic/env.py

Le script qu'Alembic exécute à chaque migration. Il précise notamment comment se connecter en mode asynchrone (`asyncpg`) et où trouver les métadonnées des modèles (`app.models`) pour comparer le schéma Python à l'état réel de la base.



python

```
```python
"""Configure Alembic pour le moteur SQLAlchemy asynchrone."""

import asyncio
import os
from logging.config import fileConfig

from alembic import context
from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config

from app.models import Base

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# L'environnement est prioritaire sur l'URL locale du fichier INI.
config.set_main_option(
    "sqlalchemy.url",
    os.getenv("DATABASE_URL", config.get_main_option("sqlalchemy.url")),
)
target_metadata = Base.metadata

def run_migrations_offline() -> None:
    """Génère le SQL sans ouvrir de connexion à PostgreSQL."""
```

```
context.configure(
    url=config.get_main_option("sqlalchemy.url"),
    target_metadata=target_metadata,
    literal_binds=True,
    dialect_opts={"paramstyle": "named"},
    compare_type=True,
)
with context.begin_transaction():
    context.run_migrations()

def run_synchronous_migrations(connection: Connection) -> None:
    """Exécute Alembic sur l'adaptateur synchrone de la connexion async."""

    context.configure(
        connection=connection,
        target_metadata=target_metadata,
        compare_type=True,
    )
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online() -> None:
    """Ouvre la connexion async et applique les migrations."""

    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
```

```
)  
async with connectable.connect() as connection:  
    await connection.run_sync(run_synchronous_migrations)  
await connectable.dispose()  
  
if context.is_offline_mode():  
    run_migrations_offline()  
else:  
    asyncio.run(run_migrations_online())  
'''
```

4.9 `alembic/script.py.mako`

Un **modèle** (template) utilisé par Alembic pour générer automatiquement la structure de chaque *future* migration (tu ne le modifies pas toi-même, il sert en coulisses).



python

```
```mako
"""${message}

Révision : ${up_revision}
Révision précédente : ${down_revision | comma,n}
Date : ${create_date}
"""

from collections.abc import Sequence
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

revision: str = ${repr(up_revision)}
down_revision: str | None = ${repr(down_revision)}
branch_labels: str | Sequence[str] | None = ${repr(branch_labels)}
depends_on: str | Sequence[str] | None = ${repr(depends_on)}

def upgrade() -> None:
 """Applique cette révision."""

 ${upgrades if upgrades else "pass"}

def downgrade() -> None:
 """Annule cette révision."""

 ${downgrades if downgrades else "pass"}
```
```

4.10 alembic/versions/__init__.py

Fichier vide, encore une fois juste pour que Python reconnaisse le dossier comme un paquet.



```
python
```python
"""Contient les révisions versionnées du schéma CMU."""
```
```

4.11 alembic/versions/20260814_0001_schema_initial.py

La migration elle-même. Une migration Alembic a toujours deux fonctions : `upgrade()` (ce qu'il faut faire pour appliquer le changement — ici, créer les 22 tables) et `downgrade()` (comment annuler ce changement — ici, les supprimer). C'est ce fichier qu'Alembic exécute à l'étape 9 du mode opératoire.



```
python
```

```
```python
"""Crée le schéma initial complet du simulateur CMU.

Révision : 20260814_0001
Révision précédente : aucune
"""

from collections.abc import Sequence

from alembic import op

from app.models import Base

revision: str = "20260814_0001"
down_revision: str | None = None
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None

def upgrade() -> None:
 """Crée les vingt-deux tables, leurs contraintes et leurs index uniques."""

 # AuditMixin injecte les quatre colonnes d'audit sans duplication.
 Base.metadata.create_all(bind=op.get_bind(), checkfirst=False)

def downgrade() -> None:
 """Supprime les tables dans l'ordre inverse de leurs dépendances."""
```

```
SQLAlchemy traite la contrainte différée du cycle facture-entente.
Base.metadata.drop_all(bind=op.get_bind(), checkfirst=False)
'''
```

## 4.12 `setup_db.sh`

Un script Bash qui automatise deux actions à la suite : créer la base PostgreSQL si elle n'existe pas encore, puis appliquer la migration (`alembic upgrade head`). Pratique pour ne pas taper les commandes une par une à chaque fois.



bash

```
```bash
#!/usr/bin/env bash
# Crée la base PostgreSQL puis applique toutes les migrations Alembic.

set -euo pipefail

# Les valeurs restent configurables sans modifier le fichier.
DB_HOST="${POSTGRES_HOST:-localhost}"
DB_PORT="${POSTGRES_PORT:-5432}"
DB_NAME="${POSTGRES_DB:-cmu_simulator}"
DB_USER="${POSTGRES_USER:-postgres}"
DB_PASSWORD="${POSTGRES_PASSWORD:-postgres}"
export PGPASSWORD="${DB_PASSWORD}"

# La vérification rend le script rejouable sans écraser une base existante.
if ! psql --host="${DB_HOST}" --port="${DB_PORT}" --username="${DB_USER}" \
  --dbname="postgres" --tuples-only --no-align \
  --command="SELECT 1 FROM pg_database WHERE datname = '${DB_NAME}'" | grep -q 1; then
  createdb --host="${DB_HOST}" --port="${DB_PORT}" \
    --username="${DB_USER}" "${DB_NAME}"
  echo "Base ${DB_NAME} créée."
else
  echo "Base ${DB_NAME} déjà présente."
fi

# Alembic reçoit la même URL que la future application.
export DATABASE_URL="postgresql+asyncpg://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}"
python -m alembic upgrade head
echo "Schéma CMU installé avec succès."
```
```

## 5. Le diagramme entité-relation, expliqué

Un diagramme entité-relation (ERD) montre visuellement les tables et leurs liens, sans avoir à lire tout le code. La notation Mermoid utilisée ici (`erDiagram`) suit une convention simple à lire :



text

```
TABLE_A ||--o{ TABLE_B : verbe
```

se lit : "une ligne de `TABLE_A` peut être liée à **zéro ou plusieurs** lignes de `TABLE_B`" (le `||` côté A veut dire "exactement une", le `o{` côté B veut dire "zéro ou plusieurs"). Le mot après les deux-points est juste une étiquette qui décrit la relation en langage naturel (ex. "reçoit", "emet").

```

```mermaid
erDiagram
    TB_REF_ASSURES ||--o{ TB_FACTURES : recoit
    TB_REF_ASSURES ||--o{ TB_ENTENTES_PREALABLES : demande
    TB_REF_CENTRES_SANTE ||--o{ TB_CENTRES_SANTE_AGENTS : affecte
    TB_REF_AGENTS ||--o{ TB_CENTRES_SANTE_AGENTS : concerne
    TB_REF_PROFESSIONNELS_SANTE ||--o{ TB_REF_PROFESSIONNELS_SANTE_SPECIALITES_MEDICALES : exerce
    TB_REF_SPECIALITES_MEDICALES ||--o{ TB_REF_PROFESSIONNELS_SANTE_SPECIALITES_MEDICALES : qualifie
    TB_REF_PROFESSIONNELS_SANTE ||--o{ TB_PROFESSIONNELS_SANTE_CENTRES_SANTE : travaille
    TB_REF_CENTRES_SANTE ||--o{ TB_PROFESSIONNELS_SANTE_CENTRES_SANTE : accueille
    TB_REF_CENTRES_SANTE ||--o{ TB_FACTURES : emet
    TB_FACTURES ||--o{ TB_FACTURES_PATHOLOGIES : documente
    TB_REF_PATHOLOGIES ||--o{ TB_FACTURES_PATHOLOGIES : caracterise
    TB_FACTURES ||--o{ TB_FACTURES_PRESCRIPTIONS : contient
    TB_FACTURES ||--o{ TB_FACTURES_PRESTATIONS : contient
    TB_REF_PROFESSIONNELS_SANTE ||--o{ TB_FACTURES_PRESTATIONS : realise
    TB_FACTURES ||--o{ TB_FACTURES_REJETS : subit
    TB_FACTURES ||--o{ TB_FACTURES_STATUTS : traverse
    TB_REF_CENTRES_SANTE ||--o{ TB_ENTENTES_PREALABLES : emet
    TB_FACTURES o|--o| TB_ENTENTES_PREALABLES : reference
    TB_ENTENTES_PREALABLES ||--o{ TB_ENTENTES_PREALABLES_STATUTS : traverse
    TB_REF_AGENTS ||--o{ TB_ENTENTES_PREALABLES_STATUTS : traite
    TB_ENTENTES_PREALABLES ||--o{ TB_ENTENTES_PREALABLES_ACTES_MEDICAUX : detaille
    TB_REF_PROFESSIONNELS_SANTE ||--o{ TB_ENTENTES_PREALABLES_ACTES_MEDICAUX : prescrit
    TB_ENTENTES_PREALABLES ||--o{ TB_ENTENTES_PREALABLES_PRESTATIONS : detaille
    TB_REF_PROFESSIONNELS_SANTE ||--o{ TB_ENTENTES_PREALABLES_PRESTATIONS : prescrit
```

```

## Le catalogue des tables, en résumé

### Catalogue des tables et clés

| Table                                             | Clé primaire                       | Principales références         |
|---------------------------------------------------|------------------------------------|--------------------------------|
| TB_REF_ASSURES                                    | PERSONNE_UUID                      | -                              |
| TB_REF_CENTRES_SANTE                              | CENTRE_SANTE_CODE                  | -                              |
| TB_REF_AGENTS                                     | AGENT_CODE                         | -                              |
| TB_CENTRES_SANTE_AGENTS                           | centre + agent + début             | centre, agent                  |
| TB_REF_PROFESSIONNELS_SANTE                       | PROFESSIONNEL_SANTE_CODE           | -                              |
| TB_REF_SPECIALITES_MEDICALES                      | SPECIALITE_MEDICALE_CODE           | -                              |
| TB_REF_PROFESSIONNELS_SANTE_SPECIALITES_MEDICALES | professionnel + spécialité + début | professionnel, spécialité      |
| TB_PROFESSIONNELS_SANTE_CENTRES_SANTE             | professionnel + centre + début     | professionnel, centre          |
| TB_REF_PATHOLOGIES                                | code + date de début               | -                              |
| TB_REF_MEDICAMENTS                                | code + date de début               | -                              |
| TB_REF_ACTES_MEDICAUX                             | code + date de début               | -                              |
| TB_TV_TYPES_FACTURES                              | code + date de début               | -                              |
| TB_FACTURES                                       | FACTURE_NUMERO                     | assuré, centre, entente        |
| TB_FACTURES_PATHOLOGIES                           | facture + pathologie + début       | facture, pathologie versionnée |
| TB_FACTURES_PRESCRIPTIONS                         | facture + prescription + début     | facture                        |
| TB_FACTURES_PRESTATIONS                           | facture + prestation               | facture, professionnel         |

| Table                                 | Clé primaire             | Principales références  |
|---------------------------------------|--------------------------|-------------------------|
| TB_FACTURES_REJETS                    | facture + rejet + début  | facture                 |
| TB_FACTURES_STATUTS                   | facture + statut + début | facture                 |
| TB_ENTENTES_PREALABLES                | ENTENTE_PREALABLE_ID     | centre, assuré, facture |
| TB_ENTENTES_PREALABLES_STATUTS        | entente + statut + début | entente, agent          |
| TB_ENTENTES_PREALABLES_ACTES_MEDICAUX | entente + acte           | entente, professionnel  |
| TB_ENTENTES_PREALABLES_PRESTATIONS    | entente + prestation     | entente, professionnel  |

Le catalogue contient **22 tables** : 12 tables de référentiel, 6 tables de facturation et 4 tables d'ententes préalables.

Colle ce diagramme dans `docs/schema_er.md` (voir étape 5 du mode opératoire) et garde-le pour ton mémoire — c'est le genre de visuel qui aide énormément un jury à comprendre la structure du projet en 30 secondes.

## 6. Vérifications détaillées (à faire après l'installation)

Reprends ces requêtes une par une pour confirmer que tout est en ordre.

**Révision Alembic active :**



```
bash
```

```
python -m alembic current
```

```
→ attendu : 20260814_0001 (head)
```

**Nombre de tables :**



```
sql
```

```
SELECT COUNT(*) AS nombre_tables
FROM information_schema.tables
WHERE table_schema = 'public' AND table_name LIKE 'TB_%';
```

→ attendu : 22

**Colonnes d'audit présentes partout :**



```
sql
SELECT table_name, COUNT(*) AS nombre_colonnes_audit
FROM information_schema.columns
WHERE table_schema = 'public'
 AND table_name LIKE 'TB_%'
 AND column_name IN ('DATE_CREATION','UTILISATEUR_ID_CREATION',
 'DATE_MODIFICATION','UTILISATEUR_ID_MODIFICATION')
GROUP BY table_name
ORDER BY table_name;
```

→ attendu : 22 lignes, chacune avec 4

**Générer le SQL sans toucher la base (pour ton mémoire) :**



```
bash
python -m alembic upgrade head --sql > schema_initial.sql
```

**⚠ Ne jamais faire sur une base contenant de vraies données :**



```
bash
python -m alembic downgrade base # supprime TOUTES les tables de l'étape !
```

## 7. Dépannage — les erreurs les plus courantes

| Message d'erreur                                      | Cause probable                                                                | Solution                                                                                                 |
|-------------------------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>connection refused</code>                       | PostgreSQL n'est pas démarré, ou mauvais port                                 | Démarre PostgreSQL, vérifie le port 5432 et le pare-feu                                                  |
| <code>password authentication failed</code>           | Mauvais utilisateur/mot de passe                                              | Corrige <code>POSTGRES_USER</code> / <code>POSTGRES_PASSWORD</code> ou <code>DATABASE_URL</code>         |
| <code>createdb</code> / <code>psql</code> introuvable | Le dossier <code>bin</code> de PostgreSQL n'est pas dans le <code>PATH</code> | Ajoute-le (ex. <code>C:\Program Files\PostgreSQL\15\bin</code> )                                         |
| <code>ModuleNotFoundError: app</code>                 | Alembic lancé depuis le mauvais dossier                                       | Lance les commandes depuis la racine du projet                                                           |
| Migration interrompue en cours de route               | Erreur PostgreSQL au milieu de l'exécution                                    | Lis l'erreur, corrige la cause, vérifie <code>alembic current</code> , relance <code>upgrade head</code> |

## 8. Ce qui reste à faire (pas dans cette étape)

Les données de référence et les scénarios simulés → **étape 2**.

Les droits ouverts et les cotisations restent à définir plus tard.

La règle de validation automatique après 4 heures → couche métier future.

Les schémas Pydantic, endpoints FastAPI, gestion d'erreurs → **étape 5**.

Socket.IO, KPI temps réel et React → **étapes 4 et 6**.

**Fin du guide expliqué de l'étape 1.**